

Mindanao State University - Iligan Institute of Technology
College of Computer Studies
Department of Computer Applications
BCA182 Embedded Systems Programming

Last Name: _____

Date: _____

First Name: _____

Subject/Section: _____

Laboratory Activity 2
Personal MP3 Player
50 Points

Contents

1. Introduction	2
1.1 Laboratory Overview	2
3. Hardware Setup	3
3.1 Board Layout	3
4. Final Project.....	5
4.1 Song_def.h.....	5
4.2 Timeout vs Ticker	5
4.3 Your Application Code	5

1. Introduction

1.1 Laboratory Overview

In this lab, we will remake the audio player with the following features:

- Users will be able play 8 different songs using four buttons
- Users will be able to stop and replay the song from the audio player
- Users will be able to display the name of a song that is being played in the LCD
- Users will be able to display a confirmation message when choosing the song in the LCD
- Users will have 5 seconds to confirm the song that they want to play
- Users will be able to see the state of the audio player using RGB LEDs (playing song, waiting for next song, no playing song)
- Users will display the instructions for using the audio player via UART
- Users will be able to adjust the volume of a song
- Users will be able to execute these features in three threads

By the end of this project you will have built your own “MP3 player” and you should feel confident about the techniques and concepts that you have learned during this course.

2. Requirements

In this lab, we will be using the following hardware, software, and firmware:

- **PlatformIO**
- **STM32 Framework. Arduino Framework is Prohibited.**
- **FreeRTOS or Zephyr OS**
- **RT-Thread RT-Spark Development Board**
- A breadboard, 4 x 330 Ω resistors, 4 x buttons, 1 x 100 Ω resistor, 1 x 10k Ω potentiometer, 1 x LCD, 1 x speaker (ABI-001-RC).

3. Hardware Setup

3.1 Board Layout

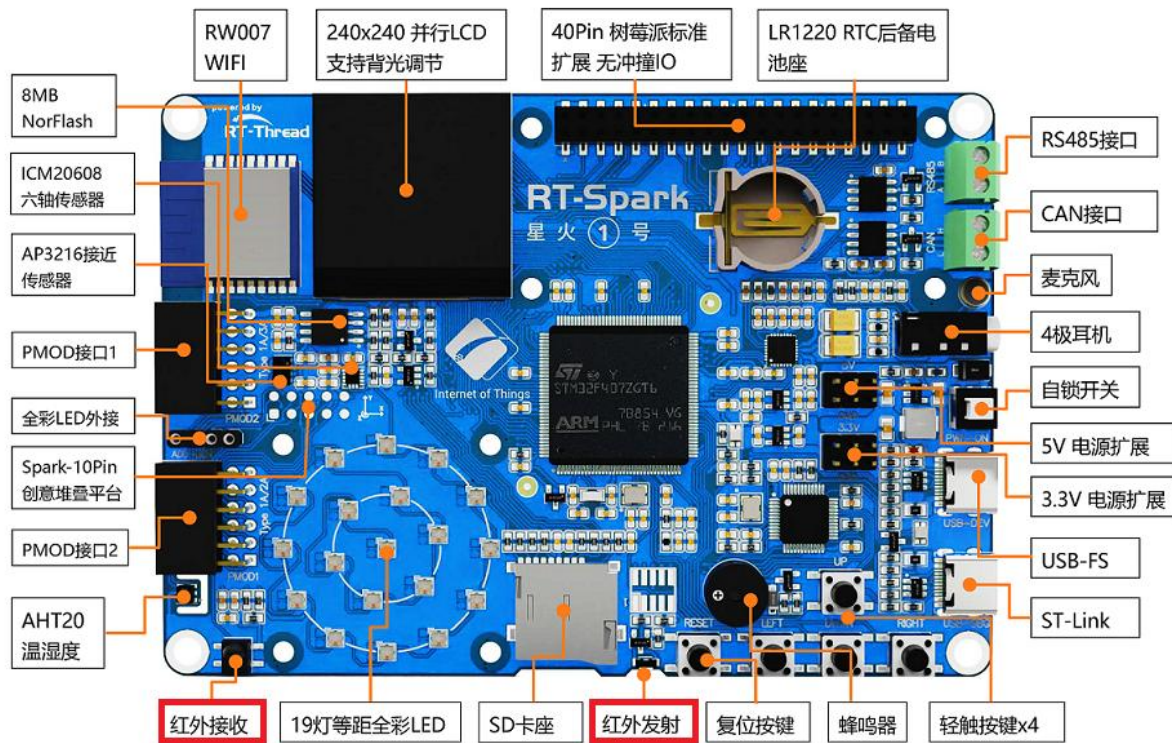


Figure 1.1 The RT-Thread Development Board Peripherals Layout

Figure 1.1 shows the RT-Spark Development Board peripheral layout, including all modules attached to the board. In this laboratory activity, you will utilize the following peripherals present in the development board: (i) 240x240 LCD on-board display, (ii) 3.5mm headphone jack to be connected into an external speaker, (iii) an external potentiometer, 5 external push buttons, (iv) three external LEDs (i.e., RED, GREEN, and BLUE LEDs), and (v) UART Pins (i.e., RX and TX on-board pins).

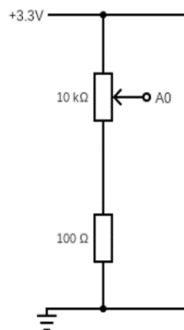


Figure 1.2 External Potentiometer Setup

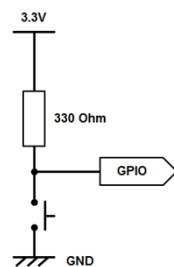


Figure 1.3 External Push Setup (using pull-up resistor setup)

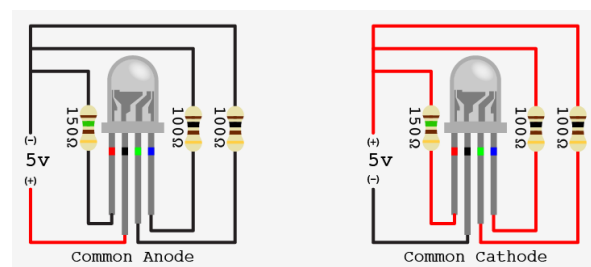


Figure 1.4 RGB LED setups

Shown in figures 1.2, 1.3, and 1.4 are the configuration of the external potentiometer, external push buttons using pull-up resistor, and the RGB LEDs setup in either common anode or common cathode setups.

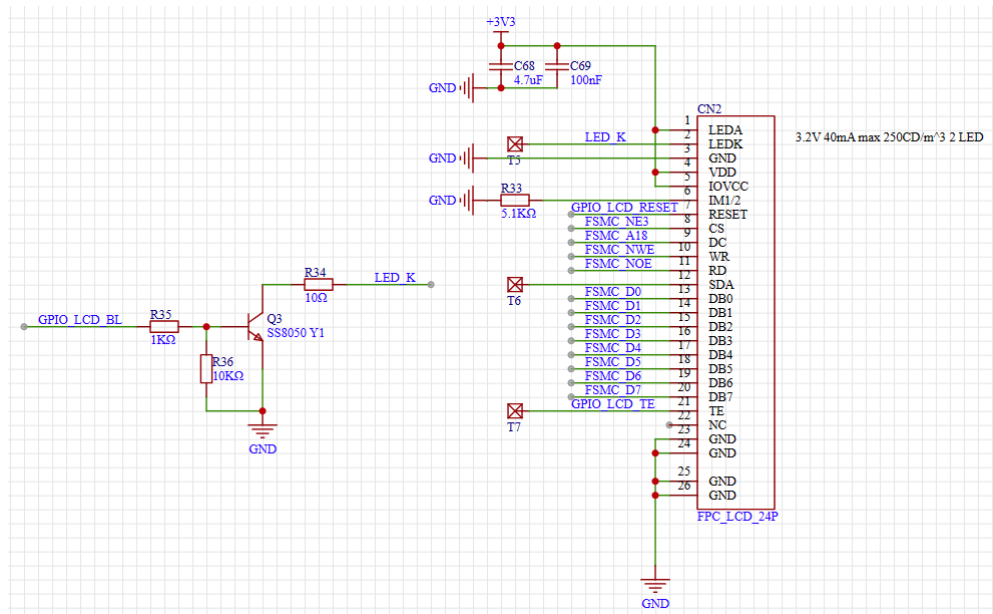


Figure 1.4 RT-Spark LCD Connection Diagram Part 1

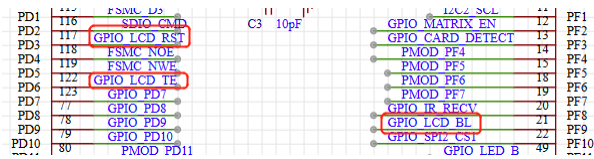


Figure 1.5 RT-Spark LCD Connection Diagram Part 2

Shown in figures 1.4 and 1.5 is a 1.3-inch LCD screen of the development board with a resolution of 240x240, providing a very sharp display. The screen uses an ST7789 v3 driver chip, and the communication interface is an 8080 parallel port. Communication with the microcontroller is achieved through FSMC simulation of the driver timings. Eight data lines are used for data transmission, and one address select line serves as the chip's enable signal.

CODEC-AUDIO

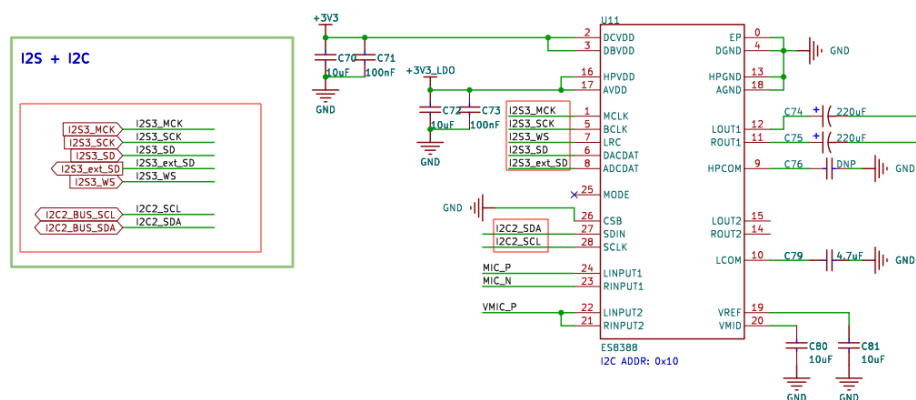


Figure 1.6 RT-Spark Audio Codec Module

Shown in Figure 1.6 is the ES8388 audio code module of the RT-Spark development board, a high-performance, low-power stereo audio codec by Everest Semiconductor, featuring 2-channel ADC/DAC, microphone amplifiers, and a headphone driver. It supports up to 96 kHz sampling rates and 24-bit resolution, commonly used in ESP32, STM32, and portable audio devices via I2S and I2C.

4. Final Project

In this project, you will build our own “MP3 player”, for the sake of this project, students do not need to generate the songs. Students should download the following files: (i) NHD_0216HZ.h (**basis only** since you will be using the on-board LCD display of the development board), (ii) NHD_0216HZ.cpp (**basis only** since you will be using the on-board LCD display of the development board), (iii) song.h, (iv) song_def.h and (v) main.cpp (**basis only** since it uses mbed OS, and you will use FreeRTOS or Zephyr OS in this project). These files should be placed in the same directory.

4.1 Song_def.h

There are a total of 10 different songs that have been provided for this lab; each of these songs has its own name (that has been split into two attributes, name1 and name2), notes, beats, tempos or speed, and length. In order to access these attributes, you can:

<SONG>.note[noteIndex] or *(<SONG>.note + noteIndex)

Note: There are no notes in the songs provided, please take care when dealing with them. It might be useful to review the module 8 lab solution to remember how to play a song.

4.2 Timeout vs Ticker

The Ticker interface is used to setup a recurring interrupt to repeatedly call a function at a specified rate whereas Timeout setup an interrupt to a call function after a specified delay. For instance, if we have the following code:

```
Ticker tickerflipper;
Timeout timeoutflipper;
DigitalOut led1(LED1);
DigitalOut led2(LED2);

void tickerFlip() {
    led1 = !led1;
}
void timeoutFlipper{
    led2 = !led2;
}
int main() {
    led1 = 1;
    led2 = 1;
    tickerflipper.attach(&tickerFlip, 2.0); // the address of the function to be attached (flip) and the interval (2 seconds)
    timeoutflipper.attach(&timeoutFlipper, 2.0); //setup flipper to call flip after 2 seconds
    // spin in a main loop.
    while(1) {
        wait(0.2);
    }
}
```

After two seconds led1 and led2 will turn off, but after another two more seconds just led1 will turn on because the tickerFlip function will execute as long as the program continues while the timeoutFlipper will just executed once, unless is setup again.

4.3 Your Application Code

In this exercise, you are required to create your own main.cpp using STM32CubeMX, STM32cubeIDE, and FreeRTOS or Zephyr RTOS, and utilize the provided main.cpp to perform the following requirements

below:

- Define inputs and outputs that are missing.
- Complete the functions `update_lcd_leds_thread()`, `adjust_volume()` and `polling_buttons()`.
- Use buttons 2 – 4 to insert a song in binary form from the array of songs. For instance, if the all the buttons are low or they are zero then we would end up playing the first song.
- While the buttons (2-4) are being pressed, press button 1 to select a song and then release all the buttons.
- Press button 1 again to confirm your choice, if you don't do anything for 5 seconds, everything will continue normally.
- Display name of the song on the LCD when playing and a confirmation message when changing the song.
- Use the timeout and ticker interface to wait for the 5 seconds of confirmation and play music respectively.
- Use `USER_BUTTON` to stop/play a song.
- When a song is played the blue LED should be on
- When a song is paused the red LED should be on
- When changing a song the green LED should be on
- Create 3 threads to:
 1. Update the LCD and RGB LEDs
 2. Polling buttons
 3. Adjust the volume
- **Note that:**
 - Every thread contains an infinite loop
 - Tasks are scheduled in a cooperative manner, namely, threads can go to waiting state after each update, to let the next thread be scheduled
 - Since the LCD must be accessed exclusively, we need a mutex to protect its single access.
- In the main program:
 - Clear the LCD display
 - Start all threads
 - Send the instruction of how to use the audio player via UART
 - Use sleep mode to reduce power consumption